AionFM: A Hybrid Continuous-Discrete Temporal Foundation Model for Probabilistic, Regime-Aware Time-Series Forecasting

Open Technical Specification

Marlon Hanks

Independent Researcher

May 2026

Conceptual Technical Specification — Open for Community Development

---

Abstract

This paper introduces **AionFM**, a hybrid continuous-discrete time-series foundation model that unifies zero-shot forecasting, probabilistic scenario generation, regime detection, and domain adaptation within a single decoder-only architecture. AionFM processes temporal data through two synchronized representational streams: a **continuous value patch stream** that preserves full numerical precision and a **discrete regime token stream** that encodes behavioral abstractions including trend direction, volatility dynamics, seasonality phase, discontinuity structure, missingness patterns, and cross-variable structural states. Central to the architecture is the concept of a **residual temporal language** — rather than tokenizing raw observations directly, the model first estimates a coarse future trajectory from local context, then models uncertainty and local behavioral shifts through symbolic residual states, and finally refines predictions with a conditional quantile-residual decoder. This residual abstraction enables cross-domain transfer because behavioral concepts such as "volatility expansion," "shock recovery," and "mean-reverting compression" recur across finance, energy, retail, logistics, healthcare, and industrial telemetry even when raw

magnitudes differ by orders of magnitude. The pretraining framework combines seven complementary objectives: next-patch prediction, masked temporal reconstruction, regime-token likelihood, quantile loss, rollout consistency, cross-scale contrastive alignment, and calibration loss — applied across heterogeneous real and synthetic time-series corpora that span diverse frequencies, variable counts, and domain semantics. At inference, the model produces point forecasts, calibrated quantile intervals, and multiple plausible future trajectories from a single historical context, with optional lightweight adapters for exogenous variables, calendar structure, entity metadata, or sensor descriptors. AionFM is applicable across finance, energy, retail, logistics, mobility, healthcare operations, climate, and industrial telemetry without task-specific architectural redesign, operating simultaneously as a point forecaster, probabilistic forecaster, scenario generator, regime detector, imputer, and domain-adaptable forecasting engine.

Table of Contents

1. **1. Introduction**
2. **2. Motivation and Problem Statement**
3. 2.1 Limitations of Statistical Forecasting
4. 2.2 Limitations of Deep Learning Approaches
5. 2.3 Limitations of Current Foundation Model Approaches
6. 2.4 The Regime Modeling Gap
7. **3. Design Philosophy**
8. **4. The Residual Temporal Language**
9. **5. Model Architecture**
10. 5.1 System Overview
11. 5.2 Input Representation
12. 5.3 Normalization Strategy

13. 5.4 Continuous Value Patch Encoder
14. 5.5 Discrete Regime Tokenizer
15. 5.6 Dual-Stream Causal Backbone
16. 5.7 Temporal Position Encoding
17. 5.8 Memory Mechanism for Long Context
18. **6. Training Framework**
19. 6.1 Composite Objective
20. 6.2 Pretraining Data Strategy
21. 6.3 Synthetic Trajectory Generator
22. **7. Probabilistic Forecasting and Scenario Generation**
23. 7.1 Output Heads
24. 7.2 Forecast Decomposition
25. 7.3 Two-Stage Scenario Generation
26. 7.4 Example Output Schema
27. **8. Domain Adaptation and Deployment**
28. 8.1 Adaptation Modes
29. 8.2 Covariate Integration
30. 8.3 Model Family
31. 8.4 Deployment Modes
32. **9. Application Domains**
33. **10. Evaluation Framework**
34. 10.1 Point Forecast Metrics
35. 10.2 Probabilistic Metrics
36. 10.3 Scenario Quality Metrics
37. 10.4 Robustness Metrics
38. 10.5 Transfer Metrics

39. 10.6 Interpretability Metrics
40. 10.7 Benchmarking Protocol
41. **11. Advanced Capabilities**
42. 11.1 Constraint-Aware Forecasting
43. 11.2 Hierarchical Forecasting
44. 11.3 Retrieval-Augmented Temporal Forecasting
45. 11.4 Multivariate and Cross-Series Modeling
46. **12. Discussion and Limitations**
47. 12.1 Key Limitations
48. 12.2 Failure Modes
49. 12.3 Governance and Safety
50. **13. Future Research Directions**
51. **14. Conclusion**
52. **References**

1. Introduction

Time-series forecasting underpins decision-making across virtually every quantitative domain. Inventory planners require demand projections weeks ahead. Energy grid operators need load forecasts at sub-hourly resolution. Financial risk managers depend on volatility estimates across multiple horizons. Healthcare administrators must anticipate patient surges days before they materialize. In every case, the operational value of a forecast depends not only on the accuracy of a single point prediction but on the quality of the uncertainty envelope around it, the plausibility of alternative future trajectories, and the ability to detect when the underlying process has shifted into a fundamentally different behavioral regime. Yet the vast majority of forecasting systems deployed in practice are optimized for narrow settings — one dataset, one domain, one horizon, one frequency, or

one business objective. Single-point forecasts are fundamentally inadequate for inventory planning, risk control, or capacity management where calibrated uncertainty matters more than precise point estimates. The cost of an under-predicted demand spike is qualitatively different from the cost of an over-prediction, and systems that do not communicate this asymmetry leave decision-makers without the information they need.

The emergence of foundation models — large-scale pretrained architectures that transfer to diverse downstream tasks — has transformed natural language understanding and computer vision over the past several years. Time-series forecasting represents the next frontier for this paradigm. Several architectural approaches have emerged in the recent literature, each embodying a distinct design philosophy. **Decoder-only architectures** that process patched continuous inputs have demonstrated that pretraining at scale on heterogeneous time-series corpora yields zero-shot performance competitive with supervised baselines, leveraging subseries-level patch representations to capture local temporal structure. **Encoder-decoder architectures** that tokenize continuous values into fixed discrete vocabularies have repurposed language modeling machinery for time series, treating forecasting as a sequence-to-sequence translation problem over quantized bins. **Encoder-based universal forecasters** with any-variate attention have addressed the multivariate setting by inverting the standard attention dimension, applying attention across variables rather than across time steps. **Decoder-only probabilistic models** using lagged features and distributional output heads have combined autoregressive generation with explicit distributional assumptions. Each of these approaches represents meaningful progress, and collectively they have established that pretrained temporal models can generalize across datasets, frequencies, and domains.

However, each existing paradigm makes a fundamental architectural trade-off that limits its scope. Architectures that tokenize continuous values into discrete bins lose numerical precision: the reconstruction error introduced by quantization is non-trivial when small magnitude differences carry operational significance — a **0.3%** error in a wholesale electricity price forecast can translate to millions of dollars in hedging cost. Architectures

that preserve continuous numerical values through patch-based representations lack explicit behavioral abstraction: the model must implicitly learn regime structure from raw patch embeddings, without any dedicated mechanism for reasoning about process states such as trend reversals, volatility regime changes, or structural breaks. Encoder-based multivariate forecasters that attend across variables do not model regime structure as a first-class representational object; cross-variable correlations are captured but the notion that a system can transition between qualitatively distinct behavioral states is not architecturally encoded. No existing approach combines continuous numerical precision with discrete behavioral reasoning in a single unified architecture.

AionFM is built on the thesis that effective forecasting requires both numerical precision and behavioral abstraction — modeling time series simultaneously at a numerical level and at a behavioral level. The key insight is that many forecasting failures arise not from insufficient model capacity or inadequate training data, but from the absence of explicit representational structure for the behavioral dynamics that govern how time-series evolve. A demand series that has entered a promotional uplift regime behaves differently from one in post-holiday normalization. A sensor reading exhibiting gradual drift requires different forecasting logic than one experiencing abrupt level shifts. A financial return series in a low-volatility compression phase has different distributional properties than the same series during a volatility explosion. These distinctions are behavioral, not numerical, and they recur across domains in structurally similar forms.

To operationalize this insight, AionFM introduces a **dual-stream architecture** that processes time series through two synchronized representational pathways. The **continuous value patch stream** segments normalized observations into subseries-level patches and projects them into dense embeddings, preserving full numerical resolution throughout the forecasting pipeline. The **discrete regime token stream** computes behavioral feature vectors from local temporal summaries — capturing direction, curvature, volatility, seasonality phase, shock magnitude, recovery rate, and cross-variable structural states — and quantizes these into a learned vocabulary of residual regime tokens.

Critically, the regime tokens do not encode raw observation values; they encode **residual behavior** relative to a locally estimated baseline, creating a domain-independent behavioral language that transfers across scales, frequencies, and application areas.

The result is an architecture that operates simultaneously as a **point forecaster**, **probabilistic forecaster**, **scenario generator**, **regime detector**, **imputer**, **synthetic trajectory generator**, and **domain-adaptable forecasting engine** — all within a single set of shared parameters. This paper presents the complete technical specification of AionFM, detailing the residual temporal language, the dual-stream architecture, the composite training objective, the probabilistic scenario generation mechanism, the evaluation framework, and the domain adaptation strategy. The principal contributions are as follows: **(1)** the residual temporal language, which tokenizes behavioral dynamics rather than raw values; **(2)** a dual-stream continuous-discrete architecture that preserves numerical precision while enabling behavioral reasoning; **(3)** a hierarchical regime token structure operating at macro, meso, and micro temporal resolutions; **(4)** a seven-component composite training objective spanning prediction, reconstruction, regime likelihood, quantile calibration, rollout consistency, cross-scale alignment, and calibration; **(5)** probabilistic scenario generation via two-stage regime-conditioned decoding; and **(6)** a comprehensive evaluation framework covering point accuracy, probabilistic calibration, scenario quality, robustness, transfer, and interpretability.

2. Motivation and Problem Statement

2.1 Limitations of Statistical Forecasting

The foundations of modern time-series analysis rest on a family of statistical methods developed over several decades. Exponential smoothing methods provided recursive estimators for level, trend, and seasonality, forming the basis for a taxonomy of **30** distinct model configurations spanning additive, multiplicative, and damped specifications (Hyndman and Athanasopoulos, 2021). Autoregressive integrated moving average models

established systematic procedures for model identification, estimation, and diagnostic checking, formalizing the iterative cycle of specification, fitting, and residual analysis that remains the canonical workflow in applied time-series econometrics (Box et al., 2015). State-space formulations unified exponential smoothing and autoregressive models within a single probabilistic framework, enabling maximum likelihood estimation, Kalman filtering, and smoothing across the full family of linear Gaussian models (Durbin and Koopman, 2012). Structural time-series models decomposed observed series into interpretable components — level, slope, seasonal, cycle, and irregular — facilitating both explanation and forecasting. Decomposition approaches combining piecewise linear trends, Fourier-based seasonality, and automatic changepoint detection have been widely adopted in large-scale operational settings where analyst intervention is impractical.

The strengths of these methods are well established: interpretability, theoretical guarantees regarding consistency and efficiency under model assumptions, computational efficiency that scales to millions of series, and graceful degradation under data scarcity. However, their limitations are equally well documented. Stationarity assumptions — either in levels or after differencing — are routinely violated in practice. Manual feature engineering is required to handle calendar effects, promotional uplifts, and domain-specific covariates. Nonstationarity, regime changes, and structural breaks are handled through ad hoc procedures (differencing, piecewise specification, intervention variables) rather than through principled latent-state modeling. High-dimensional dependencies among related time series are typically ignored or addressed through post hoc reconciliation rather than joint modeling. Perhaps most critically, there is no mechanism for cross-domain transfer: a model fitted to electricity demand series contains no reusable knowledge applicable to retail sales or sensor telemetry, even when the underlying temporal dynamics — trend, seasonality, level shifts — share structural similarities.

2.2 Limitations of Deep Learning Approaches

Recurrent neural networks with gated mechanisms — particularly those employing input, forget, and output gates to regulate information flow through hidden state vectors — captured temporal dependencies through hidden state propagation and achieved strong results on sequence modeling benchmarks (Hochreiter and Schmidhuber, 1997). Temporal convolutional networks offered parallelizable alternatives that applied dilated causal convolutions over the time dimension, achieving competitive accuracy with substantially faster training. The Transformer architecture (Vaswani et al., 2017), originally developed for natural language processing, was subsequently adapted for time-series forecasting with a range of innovations addressing the specific challenges of temporal data: efficient attention mechanisms for long sequences, explicit series decomposition within the architecture, frequency-domain attention formulations, and invertible normalization layers. These adaptations addressed the quadratic computational cost of standard self-attention and the absence of sequential inductive bias in the original formulation.

Patch-based representations marked a critical advance in temporal deep learning. Segmenting time series into subseries-level patches — fixed-length windows of consecutive observations — dramatically improved long-term forecasting accuracy while simultaneously enabling self-supervised pretraining analogous to masked language modeling. The patch abstraction reduces sequence length by a factor equal to the patch size, improving computational efficiency, and creates semantically richer input tokens that encode local slopes, short-term variation, and magnitude information. Inverting the attention dimension to apply attention across variables rather than across time steps proved effective for capturing multivariate correlations, particularly in settings with large numbers of co-evolving series. AionFM builds on the patch-based paradigm, extending it with discrete behavioral tokenization to add a second representational layer that the patch stream alone does not provide.

Despite these architectural innovations, deep learning approaches to time-series forecasting have historically been limited by their task-specific nature. Models are designed, trained, and evaluated on individual datasets; architectural choices are tuned to

specific forecasting horizons, frequencies, and domain characteristics. Zero-shot transfer — applying a model trained on one corpus to produce useful forecasts on entirely unseen series — was not a design objective for these approaches, and their performance degrades substantially outside the distributional support of their training data. No mechanism exists for transferring learned temporal knowledge from one domain to another.

2.3 Limitations of Current Foundation Model Approaches

The application of the foundation model paradigm to time-series forecasting has produced several distinct architectural families, each addressing a different facet of the generalization challenge. **Decoder-only patched-input architectures** pretrained at scale on large heterogeneous corpora have demonstrated that zero-shot forecasting performance can match or exceed supervised baselines trained on the target dataset, validating the premise that temporal patterns generalize across domains. However, these architectures lack explicit behavioral abstraction: the model must discover regime structure, changepoints, and process-state transitions implicitly from raw patch embeddings, without any dedicated representational mechanism for reasoning about qualitative behavioral shifts. **Encoder-decoder architectures** that tokenize continuous values into discrete vocabularies repurpose the full machinery of language modeling — embedding layers, cross-attention, autoregressive decoding — for time series, but they introduce reconstruction error through value quantization. When values are mapped into bins, the model can only predict bin identifiers, and the granularity of the bin grid imposes a hard ceiling on numerical precision. **Encoder-based universal forecasters** handle arbitrary numbers of input variables through any-variate attention mechanisms but do not model regime structure as a first-class representational object; the model attends across variables and time steps but has no dedicated pathway for detecting, representing, or forecasting behavioral state transitions. Despite the substantial progress represented by each of these approaches, no existing architecture combines continuous numerical precision with discrete behavioral regime tokenization — the representational gap that AionFM is designed to fill.

2.4 The Regime Modeling Gap

The idea that time-series dynamics shift between discrete latent states has a long history in econometrics and signal processing. **Markov-switching autoregressive models** formalized this insight by positing that the parameters of an autoregressive process are governed by a hidden Markov chain, with each state of the chain corresponding to a distinct dynamical regime (Hamilton, 1989). The framework was originally applied to business-cycle analysis, distinguishing expansion and contraction phases, but has since been extended to volatility modeling, interest rate dynamics, and exchange rate behavior. Threshold autoregressive and smooth transition autoregressive models provided alternative formulations in which regime transitions are triggered by observable variables exceeding thresholds rather than by latent Markov states. Change-point detection methods, including **Bayesian online changepoint detection** (Adams and MacKay, 2007), address structural breaks in real time by maintaining a posterior distribution over the time since the last changepoint, enabling sequential updating as new observations arrive.

Despite their theoretical elegance, classical regime-switching models suffer from key limitations that prevent their integration into modern forecasting systems. The number of regimes must be specified in advance, and model selection procedures for choosing this number are often unreliable. State definitions are hand-crafted, reflecting the analyst's prior beliefs about which dynamical parameters should switch and which should remain constant. Transition dynamics are governed by simple parametric processes — typically first-order Markov chains with fixed transition probabilities — that cannot capture the complex, context-dependent transition patterns observed in real-world systems. Most critically, regime-switching models and deep learning forecasting architectures have developed in largely separate research communities, and their integration remains a largely unexplored territory. Regime structure and forecasting remain separate endeavors: one identifies *when* the process changed; the other predicts *what* comes next. AionFM closes this gap by unifying continuous numerical precision (via value patches), discrete behavioral

abstraction (via residual regime tokens), and probabilistic scenario generation (via regime-conditioned decoding) within a single dual-stream decoder-only architecture.

3. Design Philosophy

The architecture of AionFM is governed by six foundational design principles, each motivated by a specific failure mode observed in existing forecasting systems.

Principle 1: Preserve numerical precision. Time-series forecasting is not language modeling; small numerical differences carry operational significance. A wholesale electricity price forecast that is accurate to the nearest dollar is operationally useless when hedging contracts are priced in fractions of a cent. Quantizing continuous values into discrete bins introduces reconstruction error that compounds across forecast horizons, and the granularity of any fixed bin grid represents a hard ceiling on achievable accuracy. AionFM's continuous value patch stream maintains full metric-scale numerical information throughout the encoding, processing, and decoding pipeline, ensuring that the architecture does not impose unnecessary precision constraints on the forecasting task.

Principle 2: Learn behavioral structure. Raw numerical values do not expose process state. Two time series with entirely different scales, frequencies, and domains can share identical dynamics — a negative shock followed by exponential mean reversion — and a forecasting system that recognizes this shared structure can transfer knowledge between them. Conversely, two segments of the same series with similar numerical values can exhibit qualitatively different behavioral regimes — stable oscillation versus damped drift toward saturation — and a system that represents behavior explicitly can distinguish them. AionFM's discrete regime tokens capture behavioral dynamics independent of scale, enabling cross-domain transfer of temporal reasoning.

Principle 3: Model uncertainty as a first-class object. Point forecasts are necessary but insufficient for decision-making under uncertainty. AionFM is trained from inception to

produce calibrated quantile forecasts, prediction intervals at multiple coverage levels, distributional parameter estimates, sampled future trajectories, and regime-conditioned scenario paths. Uncertainty is not a post hoc add-on but a core architectural output.

Principle 4: Separate baseline from residual dynamics. Forecasting benefits from explicit decomposition. AionFM decomposes the prediction task into a baseline trajectory (level, trend, seasonality), a seasonal adjustment component, an event and shock component, a residual behavioral component captured by regime tokens, and an uncertainty envelope. Each component is modeled by dedicated substructures, and their composition is learned rather than manually specified.

Principle 5: Support long context efficiently. Many forecasting applications require conditioning on historical windows spanning thousands of observations — multiple seasonal cycles, prior regime episodes, and historical shock patterns. Full self-attention over such contexts is computationally prohibitive. AionFM combines local patch-level attention within a recent active window with compressed recurrent memory summaries of older history, allowing information about prior regimes, long-term trends, and historical baselines to condition current forecasts without quadratic attention cost.

Principle 6: Adapt without redesign. Different domains present different covariate structures, frequency characteristics, metadata types, and variable configurations. AionFM accommodates this heterogeneity through lightweight adapter modules that can be attached to the frozen pretrained backbone, enabling domain-specific calibration without architectural modification. The same core model processes univariate retail demand, high-dimensional industrial sensor arrays, irregular clinical measurements, and panel financial data, with adapters providing the domain-specific context that covariates, calendars, and metadata require.

4. The Residual Temporal Language

The residual temporal language is the defining contribution of AionFM and the mechanism by which the architecture achieves cross-domain transfer of behavioral knowledge. The central challenge in building a foundation model for time series is that raw observations are inherently domain-specific: a value of **42.7** means something entirely different in an electricity demand series (megawatts), a retail sales series (units), and a temperature series (degrees Celsius). If one tokenizes raw observations directly — mapping numerical values into bins or codebook entries following the vector quantization paradigm (van den Oord et al., 2017) — the resulting tokens carry domain-specific semantic content that does not transfer. A token meaning "the value is between 40 and 45" is meaningful for temperature forecasting but useless for demand forecasting.

AionFM resolves this by tokenizing **residual behavior** rather than raw values. Given a time series `x_{1:T}`, the model first estimates a local baseline `b_t = f_baseline(x_{1:t})` using a lightweight causal estimator that captures local level, trend, and dominant seasonality. It then computes a normalized residual `e_t = (x_t - b_t) / (s_t + epsilon)`, where `s_t` is a local scale estimate (e.g., local standard deviation or median absolute deviation) and `epsilon` is a small constant for numerical stability. The residual stream `e_{1:T}` is scale-free, mean-adjusted, and domain-independent: it captures how the series *deviates* from its local expected behavior, rather than what its raw magnitude is.

The residual stream is then summarized into a vector of **behavioral descriptors** computed over local windows: direction (sign and magnitude of local slope), curvature (second derivative of the smoothed residual), volatility (local variance of the residual), range expansion or contraction (ratio of recent range to historical range), seasonality phase (position within the dominant seasonal cycle), local acceleration (change in slope), shock magnitude (standardized deviation from local baseline), recovery rate (speed of return toward baseline after a shock), missingness structure (pattern and density of missing observations), cross-variable divergence (change in correlation structure among co-evolving series), correlation change (derivative of rolling pairwise correlation), and distributional asymmetry (local skewness of the residual distribution).

These behavioral descriptors are mapped through a learned quantizer to **discrete regime tokens**. A regime token does not encode "the value is 17.2"; it encodes a behavioral state such as "stable upward drift," "high-volatility expansion," "negative shock with partial recovery," "seasonal peak with residual overshoot," "flat saturation with low variance," "cross-variable decoupling," "abrupt level shift," or "mean-reverting residual compression." This abstraction is what enables cross-domain transfer: the concept of "volatility expansion following a negative shock" appears in financial return series, energy spot price series, e-commerce traffic series, hospital admission surges, and industrial sensor drift — even though the raw magnitudes, frequencies, and domains differ drastically.

Behavioral Pattern	Token Semantics	Description
Drift	Stable directional movement	Sustained positive or negative slope in residuals with low curvature
Reversal	Direction change	Sign change in local slope with high curvature, indicating trend inflection
Shock	Abrupt deviation	Large standardized residual exceeding local volatility band by multiple standard deviations
Recovery	Return toward baseline	Post-shock trajectory with exponential or linear decay toward prior level
Compression	Volatility contraction	Declining local variance and narrowing range relative to historical norms
Expansion	Volatility increase	Rising local variance and widening range, often following compression phases
Saturation	Flattening near bound	Declining slope magnitude near a local maximum or minimum with low acceleration
Seasonality Peak/Trough	Cyclic extremum	Residual phase aligned with known or inferred seasonal cycle at maximum or minimum
Regime Break	Structural discontinuity	Abrupt and persistent change in statistical properties (mean, variance, autocorrelation)
Coupling	Cross-variable alignment	Increasing correlation or co-movement among previously independent variables

Behavioral Pattern	Token Semantics	Description
Decoupling	Cross-variable divergence	Decreasing correlation or divergent trajectories among previously co-moving variables
Missing Burst	Observation gap cluster	Concentrated sequence of missing values indicating sensor failure, reporting delay, or outage
Anomalous Spike	Isolated extreme value	Single-observation deviation without sustained regime change; distinguished from structural shock by rapid return

5. Model Architecture

5.1 System Overview

AionFM comprises five major subsystems that form a sequential processing pipeline: **(1) Input normalization and temporal preparation**, which standardizes raw observations, infers frequency and calendar structure, and constructs missingness masks; **(2) Continuous value patch encoder**, which segments normalized sequences into fixed-length patches and projects them into dense embeddings preserving numerical precision; **(3) Discrete regime tokenizer**, which computes behavioral feature vectors from local temporal summaries and quantizes them into hierarchical regime tokens; **(4) Dual-stream causal temporal backbone**, a decoder-only Transformer that processes fused continuous and discrete representations under causal masking; and **(5) Probabilistic forecast and scenario decoder**, which produces point forecasts, quantile estimates, distributional parameters, regime predictions, and sampled scenario trajectories.

```
┌──────────────────────────────────────────────────────────────────────────┐
INPUT TIME SERIES DATA                        │   │   (targets, covariates,
metadata, calendar, missingness masks)    │
└──────────────────────────┬──────────────────────────────────────────────┘
│                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
[1] INPUT NORMALIZATION & PREPARATION                │   │   Reversible
scaling, frequency detection, calendar encoding,     │   │   missingness
masking, entity/domain descriptors                 │
```

```
└──────────────────────────────────────────────────────────────────────┘
│                          │                                           │
│                  ┌────────┴────────┐                                  │
│                  ▼                 ▼                   ┌──────────────┐
┌──────────────────────────┐   │  [2] CONTINUOUS      │  │  [3] DISCRETE
REGIME     │   │  VALUE PATCH ENCODER │  │  TOKENIZER              │   │
Patch segmentation, │  │  Behavioral features,    │   │  linear projection,
│  │  vector quantization,    │   │  numerical embedding │  │
macro/meso/micro tokens │   └──────────┬──────────┘
└──────────┬──────────────┘              │                            │
└──────────┬──────────────┘                          │ Gated Fusion
▼   ┌──────────────────────────────────────────────────────────────┐
│          [4] DUAL-STREAM CAUSAL TEMPORAL BACKBONE                 │   │
Decoder-only Transformer with causal masking                    │   │   Patch
attention + cross-variable attention + memory attention   │   │   Temporal
position encoding + frequency-aware positions          │   │   Compressed
long-context memory                                   │
└──────────────────────────────────────────────────────────────────────┘
│                          │
│                          ▼
┌──────────────────────────────────────────────────────────────────────┐   │
[5] PROBABILISTIC FORECAST & SCENARIO DECODER              │   │   Point
forecast head │ Quantile head │ Distribution head         │   │   Regime
forecast head │ Scenario generation head │ Imputation    │   │   Inverse
normalization & constraint projection                    │
└──────────────────────────────────────────────────────────────────────┘
```

5.2 Input Representation

AionFM accepts both univariate and multivariate time series. The full input package for a single forecasting instance comprises: **historical target values** (the series to be forecast), **known future covariates** (variables whose future values are available at forecast time, such as calendar features, planned promotions, scheduled maintenance windows, or weather forecasts), **historical covariates** (variables whose values are known only up to the forecast origin), **static metadata** (time-invariant attributes such as store type, sensor location, product category, or geographic region), **calendar features** (hour of day, day of week, week of year, month, quarter, holiday indicators, and event markers), **frequency indicators** (the sampling frequency or frequencies of the input series), **missingness masks** (binary indicators of observed versus missing values), **entity identifiers** (unique identifiers for panel data settings), **domain descriptors** (categorical or embedding-based indicators of the application domain), **sensor or asset descriptors** (for industrial and IoT applications), and **optional event indicators** (binary or categorical flags for known external events).

Data Type	Structure	Example Domains
Regular univariate	Single variable, fixed frequency	Daily retail sales, hourly electricity load, monthly GDP
Regular multivariate	Multiple co-evolving variables, fixed frequency	Multi-sensor industrial readings, multi-asset financial returns
Panel data	Many entities, same variable(s), fixed frequency	Store-level sales across retail chain, patient-level vitals
Hierarchical data	Nested aggregation structure (e.g., SKU → Category → Region)	Product hierarchies in retail, geographic hierarchies in energy
High-frequency data	Sub-minute or tick-level sampling	Financial tick data, vibration sensors, network traffic
Irregular data	Non-uniform time intervals between observations	Clinical lab results, event-triggered sensor logs, transaction data
Sparse / intermittent data	Many zero or missing values with infrequent nonzero events	Rare-item demand, failure event counts, low-volume SKUs

5.3 Normalization Strategy

Cross-domain pretraining requires a normalization strategy that removes scale and offset differences between series drawn from heterogeneous domains while preserving the information needed to reconstruct original-scale forecasts. AionFM employs a **reversible normalization** framework in which multiple normalization methods are available, the appropriate method is selected based on data characteristics, and all normalization parameters are retained for inverse mapping at the output stage. Supported normalizers include: **local z-score normalization** (subtracting local mean, dividing by local standard deviation), **robust MAD scaling** (median-centering with median absolute deviation scaling for heavy-tailed distributions), **log and Box-Cox transformations** (for positively skewed distributions and multiplicative seasonality), **relative-return normalization** (for financial and growth-rate data), **seasonal residual normalization** (removing estimated seasonal component before scaling), **entity-level scale embeddings** (learned scale

representations for panel data with heterogeneous magnitudes), and **frequency-aware scaling** (adjusting normalization window lengths based on series frequency to ensure consistent statistical estimation).

5.4 Continuous Value Patch Encoder

The continuous value path follows a patching paradigm that segments time series into subseries-level patches to create semantically richer input tokens. After normalization, sequences are divided into non-overlapping or overlapping patches of length **P** with stride **S**. Each patch `p_k = x_{(k-1)S+1 : (k-1)S+P}` is a vector of **P** consecutive normalized observations. Patches are projected into dense embeddings through a learned linear transformation: `h^v_k = W_v * p_k + b_v`, where `W_v` is a **d_model** × **P** projection matrix and `b_v` is a bias vector. Unlike pure tokenization approaches that map values into a finite discrete vocabulary, this continuous projection retains high-resolution numerical information: the full information content of the **P** observations within each patch is preserved in the **d_model**-dimensional embedding. Each patch embedding implicitly captures local slopes, short-term variation patterns, magnitude information, scale-adjusted movement characteristics, cross-variable co-movement patterns (in multivariate settings where patches span multiple variables), and missingness-aware local structure (where missing values within a patch are encoded through the missingness mask and contribute to the patch representation).

5.5 Discrete Regime Tokenizer

The discrete regime tokenizer operates in parallel with the continuous patch encoder, computing behavioral feature vectors from local temporal summaries and mapping them to discrete codes. For each patch index **k**, the tokenizer computes a behavioral feature vector `f_k` from the local residual stream, summarizing the direction, curvature, volatility, range dynamics, seasonality phase, acceleration, shock characteristics, and cross-variable structure observed in a window centered on patch **k**. A learned quantizer maps these

feature vectors to discrete codes following the vector quantization paradigm (van den Oord et al., 2017) with straight-through gradient estimation: the encoder output is replaced by the nearest codebook vector during the forward pass, and gradients are passed through the quantization step unchanged during backpropagation. The codebook is updated via exponential moving average of the encoder outputs assigned to each code.

The regime token vocabulary is organized in a **hierarchical structure** with three levels of temporal resolution:

- **Macro tokens** (broad regime): Encode the dominant behavioral mode over a multi-patch window — e.g., "upward trend," "stationary oscillation," "declining trend with volatility expansion," "structural break." Macro tokens change infrequently and capture the coarsest level of temporal structure.
- **Meso tokens** (local dynamic pattern): Encode the dynamic pattern within the current macro regime over a single-patch or few-patch window — e.g., "volatility expansion," "mean-reverting residual," "seasonal peak approach," "post-shock recovery." Meso tokens change at the pace of local behavioral shifts.
- **Micro tokens** (residual detail): Encode fine-grained residual characteristics within a single patch — e.g., "positive residual spike," "residual compression," "noise-dominated," "residual trend acceleration." Micro tokens capture the fastest-changing behavioral features.

This hierarchy enables reasoning at different temporal resolutions. A forecasting query about long-term trajectory can be informed primarily by macro tokens; a query about near-term volatility dynamics can emphasize meso tokens; a fine-grained prediction over the next few steps can leverage micro tokens for maximum precision.

5.6 Dual-Stream Causal Backbone

The backbone of AionFM is a **decoder-only causal temporal Transformer** (Vaswani et al., 2017) that processes the fused continuous and discrete representations under causal

masking, ensuring that predictions at each position depend only on information from current and past positions. At each patch index **k**, the backbone receives a composite input vector constructed from: the **continuous value embedding** `h^v_k` from the patch encoder, the **discrete regime embedding** `h^r_k` from the regime tokenizer (a sum of macro, meso, and micro token embeddings), a **time embedding** encoding temporal position and calendar structure, a **variable/entity embedding** identifying which variable and entity the patch belongs to, and an **optional covariate embedding** aggregating known future and historical covariate information. These components are combined through a **gated fusion mechanism** with learned gates `alpha_k`, `beta_k`, and `gamma_k` that dynamically weight the continuous, discrete, and covariate contributions based on the local context.

The backbone supports multiple attention patterns: **long-context patch attention** (standard causal self-attention among patches within the active window), **local temporal attention** (attention restricted to a sliding window of nearby patches for computational efficiency), **cross-variable attention** (attention across variables at the same temporal position for multivariate dependencies), **regime-token attention** (attention among regime token sequences to model regime transition dynamics), **memory attention** (attention to compressed memory states representing older historical context), **covariate-aware attention** (attention conditioned on known future covariate values), and **frequency-aware position encoding** (position representations that adapt to the sampling frequency of the input series).

5.7 Temporal Position Encoding

Temporal position encoding in AionFM combines multiple sources of positional information to provide the backbone with rich temporal context. The composite position representation includes: **absolute position** (sequential index of the patch within the input sequence), **relative position** (offset between pairs of patches for relative attention formulations), **calendar position** (hour of day, day of week, week of year, month, quarter,

and holiday indicators encoded as learned embeddings and summed), **frequency embedding** (a learned embedding for the sampling frequency of the series, enabling the model to distinguish between hourly, daily, weekly, and monthly data), **elapsed-time embedding** (for irregular time series, the elapsed time between consecutive observations, enabling the model to reason about temporal gaps), **seasonal phase embedding** (position within the dominant seasonal cycle, normalized to the range $[0, 1]$), and **forecast-horizon embedding** (during decoding, a learned embedding indicating the forecast step, allowing the model to condition its uncertainty estimates on the distance from the forecast origin).

5.8 Memory Mechanism for Long Context

Many forecasting applications require historical context spanning hundreds or thousands of observations — multiple seasonal cycles, prior regime episodes, and historical shock-recovery patterns. Full self-attention over such contexts incurs quadratic computational cost and is impractical for real-time and large-scale batch applications. AionFM addresses this through a **compressed temporal memory** mechanism organized into three tiers: an **active window** (the most recent W patches, processed with full causal self-attention at maximum resolution), a **recent memory** tier (patches from the medium-term past, summarized through a learned compression function that reduces their dimensionality while preserving key statistical properties), and a **long-term memory** tier (patches from the distant past, further compressed into a small number of memory vectors that capture the most salient historical features). The backbone attends to all three tiers, but the computational cost is dominated by the active window, with recent and long-term memory contributing only a small number of additional key-value pairs.

The compressed memory captures: long-term trend information, historical seasonal patterns and their evolution, prior shock events and their recovery dynamics, regime durations and transition patterns, recurring anomaly signatures, historical volatility bands and their distribution, entity-specific behavioral baselines, and slow-moving structural changes that unfold over timescales longer than the active window.

6. Training Framework

6.1 Composite Objective

AionFM is trained with a seven-component composite loss function, each component addressing a distinct aspect of temporal understanding. The total loss is a weighted sum: `L_total = lambda_1 * L_patch + lambda_2 * L_mask + lambda_3 * L_regime + lambda_4 * L_quantile + lambda_5 * L_rollout + lambda_6 * L_contrastive + lambda_7 * L_calibration`, where the weighting coefficients `{lambda_i}` are tuned on a validation set and may be annealed during training to prioritize different objectives at different stages.

1. Next-patch prediction loss (L_patch). The model predicts future continuous patches from historical context. This is the primary forecasting loss, trained via mean squared error or Huber loss between predicted and observed patch values. It provides the fundamental capability of autoregressive temporal prediction and grounds the model in numerical accuracy.

2. Masked temporal reconstruction loss (L_mask). Random patches within the historical window are masked (replaced with a learned mask embedding), and the model reconstructs the masked patches from surrounding context. This objective improves representation quality, robustness to missing data, imputation capability, and noise tolerance. Masking ratios between **15%** and **40%** are applied stochastically during training.

3. Regime-token likelihood loss (L_regime). The model predicts future behavioral tokens via cross-entropy over the discrete regime vocabulary. This loss trains the residual temporal language by requiring the model to anticipate not just future values but future behavioral states — whether the series will continue its current regime, transition to a new regime, or exhibit a behavioral shift at the macro, meso, or micro level.

4. Quantile loss (L_quantile). The pinball loss (Koenker and Bassett, 1978) is applied at multiple quantile levels `tau ∈ {0.01, 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99}`, training the model to produce calibrated uncertainty bands. The pinball loss is asymmetric: for a given quantile level `tau`, under-predictions are penalized proportionally to `tau` and over-predictions proportionally to `(1 - tau)`, ensuring that the predicted quantiles align with their nominal coverage levels.

5. Rollout consistency loss (L_rollout). Multi-step autoregressive forecasts are generated by feeding predicted patches back as input, and the consistency of these rolled-out forecasts with single-step predictions is enforced. This loss reduces compounding error in long-horizon forecasts and ensures that the model's multi-step and single-step predictions are coherent.

6. Cross-scale contrastive alignment loss (L_contrastive). Representations computed from short-window, medium-window, and long-window contexts are aligned using contrastive predictive coding (van den Oord et al., 2018), encouraging the model to connect local movements to broader regime context. A short-window representation of a volatility spike should align with the medium-window representation of a volatility expansion regime and the long-window representation of a regime transition period. This loss helps the model distinguish noise spikes from structural breaks by providing multi-scale contextual information.

7. Calibration loss (L_calibration). This loss penalizes systematically miscalibrated prediction intervals — intervals that are consistently too narrow (providing false confidence) or too wide (providing no useful information). Calibration is evaluated via strictly proper scoring rules (Gneiting and Raftery, 2007), which guarantee that the optimal forecast is the true conditional distribution. The calibration loss ensures that the empirical coverage of predicted intervals matches their nominal levels across the training distribution.

6.2 Pretraining Data Strategy

AionFM is pretrained on a heterogeneous mixture of time-series data spanning diverse domains, frequencies, variable counts, and temporal characteristics. The pretraining corpus is designed to expose the model to the full spectrum of temporal behaviors encountered in real-world applications. **Real data sources** include: retail sales (daily and weekly product-level demand), energy demand (hourly and sub-hourly load and generation), web traffic (page views, API calls, session counts), industrial sensors (temperature, pressure, vibration, throughput, quality metrics), weather and climate (temperature, precipitation, wind, solar irradiance), transportation (traffic flow, transit ridership, flight delays), healthcare operations (emergency department arrivals, bed occupancy, procedure volume, supply consumption), financial markets (returns, volume, implied and realized volatility, spreads), economic indicators (GDP, unemployment, inflation, leading indicators), manufacturing telemetry (cycle times, yield rates, equipment states), and supply-chain flows (orders, shipments, inventory levels, lead times). Real data provides natural complexity including human behavioral patterns, operational constraints, market feedback loops, sensor noise characteristics, and physical constraints.

Synthetic data is generated to improve coverage of rare temporal behaviors that are underrepresented in real corpora: abrupt regime changes, volatility explosions, long seasonal cycles, missing-data bursts, rare shock events, cascading failures, structural breaks, and counterfactual future paths. The synthetic generator (described in Section 6.3) produces series with known latent structure, providing direct supervision for the regime tokenizer and enabling evaluation of interpretability. The ratio of real to synthetic data and the domain mixture weights are treated as hyperparameters and tuned to maximize zero-shot transfer performance on held-out validation domains.

6.3 Synthetic Trajectory Generator

The synthetic trajectory generator produces time series with fully specified latent structure, enabling both training and evaluation of the regime tokenizer and the broader architecture.

Each synthetic series is composed as: `x_t = T_t + S_t + C_t + E_t + R_t + epsilon_t`, where `T_t` is a trend component, `S_t` is a seasonal component, `C_t` is a cyclical component, `E_t` is an event or shock component, `R_t` is a regime-switching component, and `epsilon_t` is a noise term. The generator can simulate: **linear, piecewise linear, exponential, and logistic trends**; **single and multiple seasonalities** with configurable periods, amplitudes, and phase shifts; **random shocks** with configurable magnitude, sign, and recovery dynamics; **step changes** in level, trend, or variance; **mean-reverting processes** with configurable speed and equilibrium; **heteroskedasticity** with GARCH-like volatility clustering; **autoregressive noise** of specified order; **sparse demand patterns** with intermittent zeros and count-valued observations; **irregular sampling** with non-uniform inter-observation intervals; **sensor drift** with slow, systematic bias accumulation; and **correlated multivariate systems** with time-varying correlation structures. Critically, the latent regime labels generated during synthesis are retained and used for direct supervision of the regime tokenizer, providing ground-truth behavioral state sequences for training and evaluation.

7. Probabilistic Forecasting and Scenario Generation

7.1 Output Heads

AionFM produces forecasts through six specialized output heads, each addressing a distinct inferential need:

53. **Point forecast head** — produces a deterministic mean or median forecast, optimized for point accuracy metrics. This head applies a linear projection from the backbone's final hidden state to the forecast dimension, followed by inverse normalization to recover original-scale predictions.

54. **Quantile forecast head** — produces calibrated quantile estimates at multiple levels (e.g., **1st, 5th, 10th, 25th, 50th, 75th, 90th, 95th, 99th** percentiles), enabling construction of prediction intervals at arbitrary coverage levels. Quantile non-crossing is enforced through monotonic output constraints.
55. **Distribution head** — optionally produces parameters of a parametric distribution (Gaussian, Student-t, negative binomial, log-normal, mixture of Gaussians, zero-inflated Poisson, or zero-inflated negative binomial), enabling likelihood-based evaluation and sampling.
56. **Regime forecast head** — predicts future behavioral tokens at each forecast step, providing an interpretable characterization of the expected future behavioral trajectory (e.g., "continued stable growth for 5 steps, then seasonal decline for 3 steps").
57. **Scenario generation head** — samples multiple plausible future trajectories for risk simulation, stress testing, capacity planning, and decision support (described in detail in Section 7.3).
58. **Imputation head** — predicts missing values within historical windows, enabling the model to serve as a temporal imputation engine in addition to a forecaster.

7.2 Forecast Decomposition

Each forecast produced by AionFM is accompanied by an interpretable decomposition into constituent components: `x_hat_{t+h} = baseline_{t+h} + seasonal_{t+h} + event_{t+h} + residual_{t+h}`, where `baseline_{t+h}` captures the estimated level and trend at horizon **h**, `seasonal_{t+h}` captures the seasonal contribution, `event_{t+h}` captures the effect of known events or shocks, and `residual_{t+h}` captures the remaining behavioral component informed by regime tokens. This decomposition is learned through auxiliary heads during training rather than manually specified, and it provides forecasters and decision-makers with transparency into the

drivers of each prediction. The uncertainty envelope is additionally decomposed into contributions from each component, enabling users to understand whether forecast uncertainty is driven primarily by trend uncertainty, seasonal uncertainty, event uncertainty, or residual regime uncertainty.

7.3 Two-Stage Scenario Generation

Scenario generation is a core capability of AionFM, distinguished from simple quantile forecasting by its ability to produce temporally coherent future trajectories rather than marginal distributions at each horizon. The scenario generation process operates in two stages. In **Stage 1 (regime path sampling)**, the regime forecast head samples plausible future regime token sequences from the conditional distribution over behavioral states. Each sampled sequence represents a distinct qualitative future: one sample might encode "continued stable growth for 8 steps followed by seasonal decline," another might encode "shock event at step 3 with partial recovery," and a third might encode "volatility expansion with trend reversal at step 5." The diversity of sampled regime paths is controlled by a temperature parameter. In **Stage 2 (value decoding)**, the continuous value decoder generates numerical forecasts conditioned on each sampled regime path, producing concrete trajectories that are consistent with the behavioral narrative specified by the regime tokens.

Users can request specific scenario types: **conservative** (regime paths with high continuation probability), **median** (most likely regime trajectory), **optimistic** (paths with positive drift and compression), **tail-risk** (paths with low probability but high impact), **stress** (paths explicitly conditioning on adverse regime transitions), or **regime-specific** (paths conditioning on a user-specified behavioral state). The critical distinction between scenario generation and marginal quantile forecasting is that scenarios preserve **temporal coherence**: each trajectory tells a consistent story across the full forecast horizon, whereas marginal quantiles at different horizons may correspond to contradictory underlying dynamics.

7.4 Example Output Schema


```
{   "model_version": "AionFM-M v1.0",   "forecast_origin": "2026-05-
01T00:00:00Z",   "forecast_horizon": 14,   "frequency": "daily",   "target":
"store_42_sku_1087_units",   "point_forecast": [     142, 138, 155, 161, 170,
168, 152,     145, 140, 158, 164, 173, 171, 155   ],   "quantiles":
{     "q01": [102, 97, 108, 112, 118, 115, 105, 100, 96, 110, 114, 120, 118,
108],     "q05": [112, 107, 119, 124, 131, 128, 117, 111, 107, 121, 126, 133,
131, 119],     "q10": [119, 114, 127, 132, 140, 137, 125, 118, 114, 129, 134,
142, 140, 127],     "q25": [130, 125, 140, 146, 154, 152, 138, 131, 126, 143,
148, 157, 155, 140],     "q50": [142, 138, 155, 161, 170, 168, 152, 145, 140,
158, 164, 173, 171, 155],     "q75": [155, 151, 170, 177, 187, 184, 167, 159,
154, 173, 180, 190, 188, 170],     "q90": [168, 164, 185, 192, 203, 200, 181,
172, 167, 189, 196, 207, 204, 185],     "q95": [176, 172, 194, 202, 213, 210,
190, 181, 175, 198, 206, 217, 214, 194],     "q99": [190, 186, 210, 219, 232,
228, 207, 196, 190, 215, 223, 235, 233, 211]   },   "prediction_intervals":
{     "90%": {"lower": "q05", "upper": "q95"},     "50%": {"lower": "q25",
"upper": "q75"}   },   "scenario_paths": {     "continued_growth": [145, 142,
160, 168, 178, 176, 160, 153, 148, 166, 172, 182, 180, 164],
"shock_recovery":   [142, 138, 118, 105, 120, 135, 148, 145, 140, 158, 164,
173, 171, 155],     "seasonal_decline":  [142, 135, 128, 122, 118, 115, 112,
110, 108, 112, 118, 125, 130, 135],     "volatility_spike":  [142, 150, 128,
172, 145, 180, 130, 165, 140, 175, 148, 185, 155, 140]   },
"regime_probabilities": {     "stable_growth": 0.42,     "seasonal_peak":
0.27,     "shock_recovery": 0.18,     "reversal": 0.13   },   "explanation":
{     "current_regime": "stable_upward_drift with seasonal_approach",
"uncertainty_driver": "seasonal_phase_transition at horizon 7-9",
"change_point_probability": 0.23   } }
```


8. Domain Adaptation and Deployment

8.1 Adaptation Modes

AionFM supports four adaptation modes, offering a spectrum from zero additional training to full parameter updates:

59. **Zero-shot inference.** The pretrained model produces forecasts using only the historical context provided at inference time, with optional known future covariates. No domain-specific data is required for training or calibration. This mode is suitable for rapid deployment, cold-start scenarios, and domains where historical data is limited or unavailable. The residual temporal language enables meaningful zero-shot transfer because behavioral patterns learned during

pretraining — shock recovery, volatility expansion, seasonal phasing — generalize across domains.

60. **Few-shot calibration.** A small amount of domain-specific data (typically **50** to **500** observations) is used to calibrate scale parameters, quantile coverage levels, seasonal phase alignment, residual distribution shape, and covariate effects. The core model parameters remain frozen; only lightweight calibration parameters are updated. This mode is appropriate when limited domain data is available and rapid adaptation is needed.
61. **Adapter tuning.** Lightweight adapter modules — small feed-forward networks inserted at specific layers of the backbone — are trained on domain-specific data while the core model parameters remain frozen. Adapters can specialize the model for domain-specific covariate structures, frequency characteristics, distributional shapes, and regime vocabularies without modifying the shared pretrained representations.
62. **Full fine-tuning.** All model parameters are updated on domain-specific data. This mode is reserved for highly specialized or high-value applications with abundant domain data, where maximum accuracy justifies the computational cost and the risk of forgetting pretrained knowledge (mitigated through regularization and learning rate scheduling).

8.2 Covariate Integration

AionFM integrates covariates through dedicated embedding pathways that feed into the gated fusion mechanism of the backbone. **Known future covariates** include calendar features (hour, day, week, month, quarter, holiday indicators), planned promotions (timing, depth, type), scheduled maintenance windows, weather forecasts, price change schedules, and policy implementation dates. **Historical covariates** include actual weather observations, historical prices, past traffic or inventory levels, prior sensor readings, and

lagged economic indicators. **Static metadata** includes store type, geographic region, product category, sensor location, machine type, customer segment, and any other time-invariant attribute that provides contextual information. **Latent metadata** consists of entity-level descriptors inferred from historical behavior through a learned entity embedding, capturing behavioral archetypes (e.g., "high-volatility seasonal entity" or "stable trend entity with rare shocks") without requiring explicit metadata annotation.

8.3 Model Family

Configuration	AionFM-S (Edge)	AionFM-M (Enterprise)	AionFM-L (High-Accuracy)	AionFM-X (Research)
Patch length P	16	32	64	64
Max context (patches)	128	512	1024	2048
Max forecast horizon	96	384	720	1440
Max variables	16	128	512	2048
Regime token vocab	256	1024	4096	8192
Regime token levels	2 (macro/meso)	3 (macro/meso/micro)	3 (macro/meso/micro)	3 (macro/meso/micro)
Transformer depth	6	12	24	48
Attention width (d_model)	256	512	768	1024

Configuration	AionFM-S (Edge)	AionFM-M (Enterprise)	AionFM-L (High-Accuracy)	AionFM-X (Research)
Memory compression ratio	4:1	8:1	16:1	32:1
Quantile levels	5	9	9	19
Scenario count	10	50	100	500
Adapter size	32	64	128	256
Target use case	Edge, mobile, low-latency	General enterprise	High-accuracy batch	Research, high-value

8.4 Deployment Modes

AionFM supports four deployment modes optimized for different operational requirements. **Batch forecasting** processes large volumes of series on a scheduled cadence (daily, weekly, or monthly) for planning applications including demand planning, budget forecasting, capacity provisioning, and resource allocation. **Real-time inference** processes individual series or small batches with low latency (target: sub-second per series for AionFM-S, sub-5-second for AionFM-M) for monitoring, sensor anomaly detection, intraday demand adjustment, and traffic prediction. **Interactive scenario engine** enables planning teams to modify assumptions — promotional schedules, weather scenarios, capacity constraints, price changes, event shocks — and receive updated scenario-conditioned forecasts in real time, supporting collaborative decision-making. **Embedded forecasting API** provides programmatic access to all model capabilities through a structured request-response interface, enabling integration into enterprise systems, dashboards, optimization engines, and automated decision pipelines.

9. Application Domains

AionFM is designed for cross-domain application without architectural redesign. The following sections describe six representative domains, illustrating the inputs, outputs, and domain-specific value propositions.

9.1 Energy Demand Forecasting

Energy systems require forecasts at multiple temporal resolutions (sub-hourly to annual) and spatial granularities (individual meter to national grid). AionFM ingests historical load data, temperature time series, calendar and holiday schedules, weather forecasts (temperature, humidity, wind speed, cloud cover, solar irradiance), and regional metadata (population density, industrial vs. residential mix, climate zone). Outputs include peak demand prediction intervals for capacity planning, extreme-load scenarios for stress testing reserve margins, seasonal regime detection for identifying heating-cooling transition periods, anomaly identification for detecting unusual consumption patterns, and capacity-risk estimates quantifying the probability that demand will exceed available supply at each forecast horizon. The regime tokenizer is particularly valuable in energy applications for detecting transitions between heating-dominated and cooling-dominated demand regimes, identifying the onset of extreme weather-driven demand spikes, and distinguishing temporary load anomalies from structural shifts in consumption patterns.

9.2 Retail Demand Planning

Retail demand planning requires forecasts at the SKU-store-day level, often across hierarchies spanning thousands of products and hundreds of locations. AionFM ingests product-level sales history, promotional calendars (timing, depth, type, scope), price changes, store-level effects (location, format, traffic), holiday and event calendars, inventory constraint signals, and new-product metadata for cold-start forecasting. Outputs include SKU-level demand point forecasts and prediction intervals, quantile-based safety stock and reorder point recommendations, promotional uplift scenario paths (uplift

magnitude, cannibalization, pull-forward, post-promotion dip), stockout-risk forecasts at each horizon, and hierarchically reconciled forecasts ensuring consistency from SKU to category to department to store to region. The scenario generation capability enables planners to evaluate alternative promotional strategies, assess the demand impact of price changes, and stress-test supply plans against adverse demand scenarios.

9.3 Industrial Telemetry

Industrial and manufacturing systems generate high-frequency, high-dimensional time series from sensor arrays monitoring temperature, pressure, vibration, throughput, power consumption, and quality metrics. AionFM processes multivariate sensor streams with static machine and location metadata to produce predictive maintenance forecasts (time-to-failure distributions), drift detection alerts (identifying gradual sensor degradation or process shift), sensor anomaly forecasts (predicting future anomalous readings), failure-regime probability estimates (quantifying the likelihood that the system is transitioning toward a failure state), and remaining useful life estimates. The regime tokenizer captures the behavioral signatures of degradation modes — increasing vibration amplitude, gradual temperature drift, oscillation damping changes — that precede equipment failures.

9.4 Healthcare Operations

Healthcare facilities must forecast patient demand across multiple dimensions: emergency department arrivals, inpatient bed occupancy, surgical procedure volume, staffing requirements, and supply consumption. AionFM ingests historical operational data with calendar features (day of week, holiday, flu season), weather data, community health indicators, and facility metadata. Outputs include demand prediction intervals for staffing and resource allocation, surge scenarios for emergency preparedness, capacity-risk flags identifying periods when demand may exceed capacity, and calendar-adjusted projections that account for weekday-weekend patterns, holiday effects, and seasonal disease cycles. The scenario generation capability enables administrators to evaluate the operational impact of public health events, facility expansions, or staffing policy changes.

9.5 Logistics and Mobility

Logistics and transportation systems require forecasts of shipment volume, route congestion, delivery times, fleet utilization, and capacity needs. AionFM ingests historical flow data, weather conditions, calendar features, economic activity indicators, and network topology metadata to produce volume forecasts at route, hub, and network levels, delay-risk scenarios quantifying the probability and magnitude of delivery delays, capacity planning projections for fleet sizing and warehouse staffing, and disruption-sensitive forecasts that condition on known or anticipated disruptions (port closures, weather events, holiday surges). The regime tokenizer identifies recurring congestion patterns, seasonal flow shifts, and disruption-recovery dynamics.

9.6 Financial and Economic

Financial applications demand forecasts of returns, volume, volatility, liquidity, and economic indicators, with particular emphasis on tail-risk quantification and regime-aware uncertainty. AionFM processes asset returns, trading volume, realized and implied volatility measures, liquidity metrics, macroeconomic indicators, and cross-asset dependency structures to produce risk-scenario paths for portfolio stress testing, volatility forecasts with regime-conditioned uncertainty, regime probability estimates (e.g., low-volatility compression, transition, high-volatility expansion), stress paths conditioning on adverse market regimes, and tail-risk estimates quantifying the probability and magnitude of extreme outcomes. The dual-stream architecture is particularly well suited to financial applications because financial markets exhibit pronounced regime-switching behavior that is poorly captured by single-regime models.

10. Evaluation Framework

10.1 Point Forecast Metrics

Point forecast accuracy is assessed using a standard suite of metrics, each appropriate for different data characteristics and decision contexts (Hyndman and Athanasopoulos, 2021). **Mean Absolute Error (MAE)** measures the average absolute deviation between forecasts and observations, interpretable in the original units of the series. **Root Mean Squared Error (RMSE)** penalizes large errors more heavily than MAE due to the squaring operation, making it appropriate when large forecast errors are disproportionately costly. **Mean Absolute Percentage Error (MAPE)** expresses error as a percentage of the observed value, enabling comparison across series with different scales; however, MAPE is undefined for zero-valued observations and biased toward under-prediction. **Symmetric Mean Absolute Percentage Error (sMAPE)** addresses the asymmetry of MAPE by normalizing by the average of forecast and observation. **Mean Absolute Scaled Error (MASE)** normalizes MAE by the in-sample MAE of a naive seasonal forecast, providing a scale-independent measure that is well-defined for zero-valued series and directly comparable across datasets. **Weighted Absolute Percentage Error (WAPE)** computes the ratio of total absolute error to total observed values, providing an aggregate accuracy measure that weights high-volume series appropriately.

10.2 Probabilistic Metrics

Probabilistic forecast quality is evaluated across multiple dimensions. **Pinball loss** (also called quantile loss) measures the calibration of individual quantile forecasts, with the loss function asymmetrically penalizing errors based on the quantile level. **Continuous Ranked Probability Score (CRPS)** evaluates the entire predictive distribution by computing the integral of the squared difference between the predicted cumulative distribution function and the step function at the observation; CRPS generalizes MAE to probabilistic forecasts and is a strictly proper scoring rule (Gneiting and Raftery, 2007). **Prediction interval coverage** measures the empirical fraction of observations falling within predicted intervals at each nominal level (e.g., a **90%** prediction interval should contain approximately **90%** of observations). **Prediction interval width** measures the average width of prediction intervals, penalizing intervals that achieve coverage by being

excessively wide. **Calibration error** quantifies the systematic deviation between nominal and empirical coverage across quantile levels; a perfectly calibrated model has zero calibration error. **Weighted quantile loss** aggregates pinball loss across multiple quantile levels with domain-appropriate weights, providing a single summary measure of probabilistic forecast quality.

10.3 Scenario Quality Metrics

Scenario quality evaluation goes beyond marginal distributional accuracy to assess the temporal coherence and plausibility of generated future trajectories. **Path coherence** measures the temporal smoothness and internal consistency of individual scenario trajectories, penalizing paths with implausible discontinuities or physically impossible transitions. **Distributional similarity** compares the empirical distribution of scenario endpoints and path statistics to the observed distribution, using metrics such as the Wasserstein distance or maximum mean discrepancy. **Tail-event coverage** evaluates whether the scenario set includes paths that cover observed extreme events with appropriate frequency. **Regime transition realism** assesses whether the regime transitions embedded in scenario paths match the statistical properties of observed regime transitions. **Multivariate dependency preservation** measures whether cross-variable correlations and co-movement patterns are maintained across scenario trajectories. **Stress scenario plausibility** evaluates whether adversarial or tail-risk scenarios are operationally meaningful rather than merely statistically possible.

10.4 Robustness Metrics

Robustness evaluation quantifies model performance under data quality degradation and distributional perturbation. Evaluated conditions include: **missing data** (random and block missingness at varying rates), **corrupted observations** (outlier injection, additive noise, measurement error), **outliers** (isolated extreme values and sustained anomalous segments), **regime shifts** (abrupt changes in level, trend, or variance introduced at test

time), **seasonal changes** (phase shifts, amplitude changes, or frequency changes in seasonal patterns), **covariate dropout** (removal of covariates available during training), **scale shifts** (multiplicative rescaling of test series relative to training distributions), and **frequency changes** (testing at frequencies not represented in training data).

10.5 Transfer Metrics

Transfer evaluation assesses the model's ability to generalize across domains, frequencies, and entities. Dimensions include: **zero-shot performance** on datasets from domains not represented in pretraining, **few-shot adaptation efficiency** (accuracy improvement as a function of domain-specific calibration samples), **domain transfer** (performance when pretraining and evaluation domains differ), **frequency transfer** (performance on frequencies not seen during pretraining), **long-horizon generalization** (accuracy degradation as forecast horizon extends beyond training horizon), and **new-entity cold-start performance** (accuracy on entities with no historical data, using only metadata and cross-entity transfer).

10.6 Interpretability Metrics

Interpretability evaluation assesses whether the learned regime tokens provide meaningful and stable behavioral abstractions. **Regime-token stability** measures the consistency of token assignments across repeated runs and under small input perturbations. **Alignment with known change points** evaluates whether regime token transitions coincide with documented structural breaks, policy changes, or event boundaries. **Consistency of regime explanations** assesses whether the same behavioral pattern (e.g., "volatility expansion") receives the same token assignment across different series and domains. **Human expert evaluation** presents regime token sequences to domain experts and measures agreement between token-implied narratives and expert assessments. **Counterfactual sensitivity** evaluates whether modifying a single regime token in the conditioning sequence produces forecasts consistent with the behavioral meaning of the

modified token. **Component attribution fidelity** assesses whether the forecast decomposition (baseline, seasonal, event, residual) aligns with known data-generating processes in synthetic evaluation settings.

10.7 Benchmarking Protocol

The evaluation framework specifies five benchmark settings designed to assess different aspects of model capability. **Zero-shot evaluation** tests the model on datasets from domains entirely excluded from pretraining, measuring pure transfer performance. **Fine-tuned evaluation** tests performance after full domain-specific training, establishing an accuracy ceiling. **Few-shot evaluation** tests cold-start adaptation with **50**, **100**, **200**, and **500** domain-specific calibration samples. **Long-horizon evaluation** tests rollout stability at forecast horizons of **2**×, **4**×, and **8**× the training horizon. **Stress evaluation** tests performance under adversarial conditions including abrupt shocks, missing data bursts, distribution shifts, and exposure to rare regimes not well represented in training. In all settings, baseline models for comparison should include representative methods from three classes: classical statistical methods, deep learning single-task models, and existing time-series foundation models representing the current state of the art. Results should be reported with confidence intervals and statistical significance tests.

11. Advanced Capabilities

11.1 Constraint-Aware Forecasting

Many operational domains impose hard constraints on forecast values that must be satisfied for predictions to be actionable. Demand forecasts must be nonnegative. Energy generation forecasts must respect capacity limits. Inventory forecasts must obey balance equations (beginning inventory + receipts − demand = ending inventory). Financial forecasts may need to respect market hours or trading halts. Physical sensor forecasts must lie within physically plausible ranges. Cumulative quantities (total rainfall, cumulative

sales) must be monotonically nondecreasing. AionFM addresses these requirements through a **constrained decoder** that projects raw forecast outputs into the valid constraint space. Constraint types include: nonnegativity (via softplus or ReLU projection), upper and lower bounds (via clamping or sigmoid scaling), monotonicity (via cumulative softmax), conservation laws (via projection onto the constraint hyperplane), calendar-based closures (zeroing forecasts during known closure periods), and custom constraint functions defined by the user. Constraints are applied after inverse normalization to ensure that the constraint space is defined in original units.

11.2 Hierarchical Forecasting

Real-world forecasting systems frequently operate over hierarchical structures: a retail company forecasts at the SKU, category, department, store, region, and national levels; an energy utility forecasts at the meter, feeder, substation, and system levels. Forecasts produced independently at each level are typically incoherent — the sum of SKU-level forecasts does not equal the category-level forecast. AionFM produces forecasts at multiple levels of the hierarchy simultaneously and enforces consistency via reconciliation methods (Hyndman et al., 2011). The architecture supports both bottom-up reconciliation (aggregating fine-grained forecasts), top-down allocation (distributing aggregate forecasts using historical proportions), and optimal reconciliation (computing the minimum-variance linear combination of forecasts at all levels). Hierarchical constraints are integrated into the scenario generation process, ensuring that scenario paths are coherent across aggregation levels.

11.3 Retrieval-Augmented Temporal Forecasting

AionFM includes a **temporal retrieval module** that augments the forecasting context with historical windows exhibiting similar behavioral patterns. At inference time, the retrieval module identifies relevant historical analogs from: the same entity's history (prior instances of similar regimes, seasonal periods, or shock-recovery episodes), similar entities

(entities with matching metadata or behavioral archetypes), similar regime sequences (historical periods where the regime token sequence matches the current pattern), similar seasonal periods (analogous phases in prior seasonal cycles), and similar shock patterns (prior instances of shocks with comparable magnitude and context). Retrieved analogs are encoded and provided to the backbone as additional context through a retrieval attention mechanism, allowing the model to condition its forecasts on concrete historical precedents. This capability is particularly valuable for rare-event forecasting (where limited training examples are available), cold-start scenarios (where entity-specific history is insufficient), regime transition prediction (where prior transitions provide informative analogs), scenario generation (where historical analogs inform the plausibility of generated paths), and explainability (where retrieved analogs provide concrete justification for forecast behavior).

11.4 Multivariate and Cross-Series Modeling

AionFM supports rich multivariate and cross-series modeling through several mechanisms. **Cross-variable attention** enables the backbone to attend across variables at each temporal position, capturing contemporaneous dependencies and lead-lag relationships. **Correlation-regime tokens** extend the regime vocabulary to include cross-variable behavioral states such as “increasing correlation,” “divergence,” and “lead-lag reversal.” **Graph-aware adapters** incorporate known relational structure (spatial proximity, supply-chain links, organizational hierarchy) as attention biases. **Metadata-conditioned attention** uses entity metadata to modulate attention patterns, enabling the model to identify relevant cross-series information without attending to all variables equally. For very large panels with hundreds or thousands of variables, full cross-variable attention is computationally infeasible; in these settings, AionFM employs **retrieval-based context selection**, using metadata similarity and learned temporal similarity to identify the most informative subset of variables for each prediction, and attending only to this selected subset.

12. Discussion and Limitations

12.1 Key Limitations

Several important limitations of the proposed architecture should be acknowledged. Forecast quality is fundamentally bounded by data quality: if historical observations are noisy, biased, or unrepresentative of future conditions, no model architecture can compensate fully. Extreme regime shifts — events with no historical precedent in the training data — remain difficult for any model that learns from historical patterns, and AionFM is no exception. Long-horizon forecasts may become increasingly diffuse as prediction intervals widen with distance from the forecast origin, reflecting genuine uncertainty rather than model failure but potentially limiting operational utility. Poor-quality covariates (inaccurate weather forecasts, incomplete promotional calendars, missing event indicators) can mislead the model and degrade forecast quality relative to covariate-free baselines. Scenario generation requires careful validation: statistically plausible scenarios may be operationally impossible without explicit constraint encoding, and the diversity of generated scenarios depends on the quality and coverage of the pretraining data. The interpretability tokens produced by the regime tokenizer are learned approximations of behavioral states, not ground-truth causal descriptions: they capture statistical regularities in temporal behavior but do not identify causal mechanisms. Domain-specific constraints must be explicitly encoded for high-stakes applications; the model does not automatically infer physical laws or business rules. Calibration of prediction intervals must be monitored post-deployment, as distributional shift can degrade calibration over time. Finally, this paper presents a conceptual specification; empirical validation across diverse benchmarks, domains, and operational settings is needed to confirm the architectural choices proposed here.

12.2 Failure Modes

Understanding potential failure modes is essential for responsible deployment. **Unseen structural breaks** — sudden policy changes, catastrophic infrastructure failures, geopolitical events, pandemic onsets — produce temporal dynamics outside the support of the training distribution, and the model may produce forecasts that extrapolate pre-break dynamics rather than adapting to the new regime. **Poor covariate quality** — incorrect weather forecasts, missing promotional calendars, stale price data — can mislead covariate-conditioned forecasts; in such cases, the model may produce less accurate forecasts than a covariate-free baseline. **Overconfident uncertainty** may occur when the model encounters regimes not well represented in training data, producing prediction intervals that are too narrow because the model has not learned to associate the current behavioral pattern with high uncertainty. **Token misclassification** can occur with noisy or very short input histories, where the behavioral feature vector is poorly estimated and the regime tokenizer assigns an incorrect behavioral state. **Scenario implausibility** arises when generated scenario paths are statistically plausible (consistent with the learned distributional model) but operationally impossible (violating physical, regulatory, or business constraints not encoded in the model), underscoring the importance of constraint-aware decoding for high-stakes applications.

12.3 Governance and Safety

Responsible deployment of AionFM requires governance structures that extend beyond model accuracy. **Data quality controls** should include automated validation of input data integrity, detection of distributional drift between training and deployment data, and monitoring of covariate quality. **Forecast metadata and versioning** should accompany every forecast, documenting the model version, input features, normalization parameters, and confidence indicators used to produce the prediction. **Human oversight** is essential for high-impact applications: forecasts that trigger large financial commitments, capacity decisions, or safety-critical actions should be reviewed by domain experts before execution. **Uncertainty communication** should be designed for the decision context, presenting prediction intervals, scenario paths, and regime probabilities in formats that non-technical

decision-makers can interpret correctly. **Security and privacy** considerations include data minimization (retaining only the data needed for forecasting), encryption of data in transit and at rest, anonymization of entity-level data where appropriate, differential privacy mechanisms for training on sensitive data, and evaluation of memorization risk (the potential for the model to reproduce training data at inference time).

13. Future Research Directions

Five research directions are identified as priorities for extending the capabilities of AionFM beyond the current specification.

1. Better regime vocabularies. The current specification uses flat vector quantization within each hierarchical level. Future work should explore **hierarchical vector quantization** with residual codebooks, where each level of quantization captures progressively finer behavioral detail. **Human-interpretable token labeling** — automatically mapping learned codebook entries to natural-language behavioral descriptions — would improve explainability and enable domain experts to reason about model behavior. **Domain-specific token adapters** could specialize the regime vocabulary for domains with unique behavioral phenomena (e.g., circuit-breaker halts in financial markets, scheduled outage patterns in industrial systems). **Contrastive token refinement** would use contrastive objectives to ensure that behaviorally distinct time-series segments receive distinct tokens and behaviorally similar segments receive the same token. **Causal regime discovery** would move beyond correlational behavioral clustering to identify regime boundaries that correspond to causal mechanism changes.

2. Causal forecasting. Separating correlation from intervention is necessary for answering counterfactual questions: What happens to demand if the price increases by **10%**? What happens to load if a generator goes offline? What happens to production if a supplier delays by two weeks? Causal forecasting requires intervention-aware training procedures, structural causal model integration, and evaluation protocols that distinguish predictive

accuracy from causal identification. AionFM's regime token structure provides a natural interface for causal queries — interventions can be encoded as forced regime transitions — but the causal validity of such queries depends on training with appropriate causal supervision.

3. Agentic planning integration. AionFM can serve as the temporal reasoning component within autonomous planning systems that generate forecasts, simulate actions, compare scenario outcomes, optimize decisions, monitor realized outcomes against predictions, and update policies based on forecast errors. This integration requires interfaces between the forecasting model and decision optimization engines, simulation environments, and feedback loops for continuous learning.

4. Physics-aware constraints. For industrial, climate, and engineering systems, incorporating physical equations, conservation laws, and thermodynamic constraints into the forecasting architecture can improve both accuracy and physical plausibility. Physics-informed neural network techniques — encoding differential equations as soft constraints in the loss function or hard constraints in the decoder — represent a promising integration path.

5. Multimodal temporal modeling. Time series rarely exist in isolation; they are accompanied by textual event descriptions (news, maintenance logs, incident reports), images (satellite imagery, manufacturing visual inspection), geospatial data (location, topology, proximity), graph structure (supply chains, sensor networks, organizational hierarchies), and audio or vibration spectrograms. Integrating these modalities alongside the time-series backbone through cross-modal attention and multimodal fusion would enable richer temporal reasoning and more informative scenario generation.

14. Conclusion

This paper has presented the complete technical specification of **AionFM**, a proposed time-series foundation model built around the thesis that effective forecasting requires both numerical precision and behavioral abstraction. The dual-stream architecture — combining continuous value patches that preserve full metric-scale resolution with discrete residual-regime tokens that encode domain-independent behavioral states — addresses a fundamental gap in the landscape of temporal foundation models. Neither continuous-only nor discrete-only architectures capture the full information content needed for general-purpose temporal reasoning; AionFM unifies both within a single decoder-only framework.

The **residual temporal language** is the mechanism that enables cross-domain transfer: by tokenizing behavioral dynamics relative to a locally estimated baseline rather than raw observation values, the regime vocabulary encodes concepts — shock recovery, volatility expansion, seasonal phasing, regime transition — that recur across finance, energy, retail, logistics, healthcare, and industrial telemetry. The seven-component composite training objective ensures that the model learns to forecast accurately (next-patch prediction), represent robustly (masked reconstruction), reason behaviorally (regime-token likelihood), quantify uncertainty (quantile and calibration losses), maintain coherence over long horizons (rollout consistency), and connect local dynamics to broader context (cross-scale contrastive alignment).

The central promise of AionFM is to transform forecasting from a narrow prediction task — producing a single number for a single series at a single horizon — into a **general temporal reasoning layer** that produces point forecasts, calibrated uncertainty intervals, interpretable regime narratives, and coherent scenario trajectories from a single architecture. Realizing this promise requires sustained empirical validation across diverse benchmarks, operational deployment in high-stakes settings, and iterative refinement informed by real-world performance. This specification is offered as an open foundation for that community effort.

References

[1] Adams, R.P. and MacKay, D.J.C. (2007). Bayesian Online Changepoint Detection. *arXiv preprint arXiv:0710.3742*.

[2] Box, G.E.P., Jenkins, G.M., Reinsel, G.C., and Ljung, G.M. (2015). *Time Series Analysis: Forecasting and Control*, 5th edition. John Wiley and Sons.

[3] Durbin, J. and Koopman, S.J. (2012). *Time Series Analysis by State Space Methods*, 2nd edition. Oxford University Press.

[4] Gneiting, T. and Raftery, A.E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, 102(477), 359–378.

[5] Hamilton, J.D. (1989). A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle. *Econometrica*, 57(2), 357–384.

[6] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

[7] Hyndman, R.J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd edition. OTexts, Melbourne, Australia.

[8] Hyndman, R.J., Ahmed, R.A., Athanasopoulos, G., and Shang, H.L. (2011). Optimal Combination Forecasts for Hierarchical Time Series. *Computational Statistics and Data Analysis*, 55(9), 2579–2589.

[9] Koenker, R. and Bassett, G. (1978). Regression Quantiles. *Econometrica*, 46(1), 33–50.

[10] van den Oord, A., Vinyals, O., and Kavukcuoglu, K. (2017). Neural Discrete Representation Learning. *Advances in Neural Information Processing Systems*, 30, 6306–6315.

[11] van den Oord, A., Li, Y., and Vinyals, O. (2018). Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748*.

[12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

Disclaimer. *This paper presents a conceptual technical specification. No empirical results are reported. The architecture and methods described are proposed designs intended to guide implementation and invite community contribution.*